Assignment 6-2: Complete Guide

Sarsa, Q-Learning, and Expected Sarsa

This guide covers everything you need to understand and answer questions about this lab assignment. It explains the theory behind Temporal Difference (TD) learning algorithms, walks through each implementation in detail, and explains what results to expect and why.

1. Reinforcement Learning Background

1.1 The RL Framework

Reinforcement Learning (RL) involves an **agent** interacting with an **environment**. At each timestep t, the agent observes a state S_t, takes an action A_t, and receives a reward R_{t+1} while transitioning to state S_{t+1}.

The goal is to find a policy (a mapping from states to actions) that maximizes the expected cumulative reward (return). For episodic tasks with discount factor gamma:

$$G_t = R_{t+1} + \text{gamma} * R_{t+2} + \text{gamma}^2 * R_{t+3} + \ldots$$

1.2 Action-Value Functions (Q-functions)

The **action-value function** Q(s,a) estimates the expected return when taking action a in state s and following policy pi afterwards:

$$Q_{pi}(s,a) = E_{pi}[G_t \mid S_t=s, A_t=a]$$

The **optimal** action-value function Q*(s,a) satisfies the Bellman optimality equation:

$$Q^*(s,a) = \sum_{s'} P(s'|s,a) [R(s,a,s') + \text{gamma} * \max_{a'} Q^*(s',a')]$$

1.3 Temporal Difference (TD) Learning

TD methods learn directly from experience without a model of the environment. They update estimates based on **bootstrapping** — using current estimates to update other estimates. The key idea is the **TD error**:

$$\text{delta}_t = R_{t+1} + \text{gamma} * V(S_{t+1}) - V(S_t)$$

TD learning is the foundation for Sarsa, Q-Learning, and Expected Sarsa — all of which are TD control algorithms that maintain and update Q-value estimates.

2. The Cliff World Environment

Cliff World is a 4x12 gridworld from Sutton & Barto's textbook (Figure 6.4). It is a classic benchmark for comparing on-policy vs off-policy TD control methods.

Grid Layout:

(3,0) START	...	...	...	...	...	...	...	...	...	...	(3,11) GOAL
(2,0)											(2,11)
(1,0)											(1,11)
(0,0)	CLIFF	CLIFF	CLIFF	CLIFF	CLIFF	CLIFF	CLIFF	CLIFF	CLIFF	CLIFF	(0,11)

Key properties:

- **States:** 48 total (4 rows x 12 columns)
- **Actions:** 4 directions (up, down, left, right)

- **Start:** Bottom-left corner (row 3, col 0)
- **Goal:** Bottom-right corner (row 3, col 11)
- **Cliff:** Bottom row columns 1-10 — stepping here gives reward -100 and resets agent to start
- **Normal step reward:** -1 (encourages reaching goal quickly)
- **Discount:** gamma = 1 (undiscounted episodic task)

The cliff creates an interesting tension: the shortest path (along the cliff edge) is very risky because exploration might cause the agent to fall. A safer path (one row above the cliff) is longer but avoids the cliff entirely.

State Numbering:

States are numbered 0-47, typically row by row. The 48-state flat representation allows simple Q-table indexing: **q[state, action]**. The environment provides integer state observations.

3. Epsilon-Greedy Action Selection

All three algorithms in this lab use **epsilon-greedy** action selection to balance **exploration** (trying new actions to discover better strategies) and **exploitation** (using the best known action to maximize reward).

How it works:

- With probability epsilon: select a **random action** (exploration)
- With probability (1 - epsilon): select the **greedy action** = argmax_a Q(s,a) (exploitation)

Action probability distribution:

For each action a in state s under epsilon-greedy policy pi:

```
pi(a|s) = (1 - epsilon) + epsilon/|A| if a is greedy action pi(a|s) =
epsilon/|A| for all other actions
```

Implementation:

```
if self.rand_generator.rand() < self.epsilon: action =
self.rand_generator.randint(self.num_actions) # random else: action = self.argmax(current_q)
# greedy
```

The argmax with tie-breaking:

When multiple actions have the same maximum Q-value, standard argmax always picks the first one, introducing bias. The custom argmax breaks ties randomly:

```
def argmax(self, q_values): top = float("-inf") ties = [] for i in range(len(q_values)): if
q_values[i] > top: top = q_values[i] ties = [] # reset ties list if q_values[i] == top:
ties.append(i) # collect all tied actions return self.rand_generator.choice(ties) # random
choice among ties
```

4. Sarsa (On-Policy TD Control)

4.1 Core Concept

Sarsa is an **on-policy** TD control algorithm. "On-policy" means it learns the value of the policy it is actually following (the epsilon-greedy policy). The name comes from the tuple of values it uses: (S, A, R, S', A') — State, Action, Reward, next State, next Action.

4.2 Update Rule

After taking action A in state S, receiving reward R, and moving to state S' where action A' is chosen (using epsilon-greedy), Sarsa updates:

```
Q(S,A) <-- Q(S,A) + alpha * [R + gamma * Q(S',A') - Q(S,A)]
```

Where:

- **alpha** (step_size): learning rate — how much to update (0 < alpha <= 1)
- **gamma** (discount): how much future rewards matter (=1 for undiscounted tasks)
- **[R + gamma * Q(S',A') - Q(S,A)]**: the TD error — difference between target and current estimate
- **Q(S',A')**: Q-value of the ACTUAL next action chosen (epsilon-greedy)

4.3 Why It's Called On-Policy

Because A' is chosen by the SAME epsilon-greedy policy, Sarsa's Q-values converge to the value of the epsilon-greedy policy (not the optimal policy). This means Sarsa accounts for the exploration it will actually do, making it cautious near risky areas.

> In Cliff World: Sarsa 'knows' it sometimes takes random actions, so it avoids the cliff edge to prevent accidentally falling. It learns a safer (but slightly suboptimal) path.

4.4 Terminal State Update

When the episode ends (agent_end is called), there is no next state S'. The target becomes just R (since there's no future reward). The update simplifies to:

```
Q(S,A) <-- Q(S,A) + alpha * [R + 0 - Q(S,A)] (since Q(terminal) = 0)
```

4.5 Complete Implementation

```
# In agent_step: S = self.prev_state A = self.prev_action # Choose next action A' using
epsilon-greedy state = observation if self.rand_generator.rand() < self.epsilon: action =
self.rand_generator.randint(self.num_actions) else: action = self.argmax(self.q[state,:]) #
Sarsa update: uses actual next action A' self.q[S, A] += self.step_size * ( reward +
self.discount * self.q[state, action] - self.q[S, A] ) # In agent_end (terminal state): S =
self.prev_state A = self.prev_action self.q[S, A] += self.step_size * (reward - self.q[S,
A])
```

5. Q-Learning (Off-Policy TD Control)

5.1 Core Concept

Q-Learning is an **off-policy** TD control algorithm. "Off-policy" means it learns the value of the **optimal policy** regardless of which policy is being followed. It was one of the first major RL breakthroughs, developed by Watkins in 1989.

5.2 Update Rule

The KEY difference from Sarsa:

```
Q(S,A) <-- Q(S,A) + alpha * [R + gamma * max_a Q(S',a) - Q(S,A)]
```

Instead of using Q(S',A') (the actual next action's value), Q-Learning uses **max_a Q(S',a)** — the maximum Q-value over ALL actions from S'. This is the **greedy target**, always assuming we will take the best possible next action.

5.3 Comparison: Sarsa vs Q-Learning

Aspect	Sarsa	Q-Learning
Type	On-policy	Off-policy
Next action value	Q(S', A') - actual action	max_a Q(S',a) - best action
Converges to	Epsilon-greedy policy value	Optimal policy value Q*
Cliff World behavior	Safer path away from cliff	Optimal path along cliff edge
Affected by exploration	Yes - accounts for random moves	No - always assumes greedy
Typical performance	Higher avg reward during training	Lower avg reward during training

5.4 Off-Policy Intuition

Q-Learning learns the optimal Q* function even when following an exploratory policy. It's like planning ahead assuming you'll always make perfect decisions in the future, even though you know you'll sometimes explore randomly.

> In Cliff World: Q-Learning learns that the edge of the cliff is the optimal path. But because it actually follows an epsilon-greedy policy (with random actions), it sometimes falls off the cliff, getting a -100 penalty. This hurts its average training performance even though it found the 'optimal' path.

5.5 Complete Implementation

```
# In agent_step: S = self.prev_state A = self.prev_action state = observation # Choose next action (for behavior policy) if self.rand_generator.rand() < self.epsilon: action = self.rand_generator.randint(self.num_actions) else: action = self.argmax(self.q[state,:]) # Q-Learning update: uses MAX Q-value from next state (not actual action) self.q[S, A] += self.step_size * ( reward + self.discount * np.max(self.q[state,:]) - self.q[S, A] ) # In agent_end (terminal state): S = self.prev_state A = self.prev_action self.q[S, A] += self.step_size * (reward - self.q[S, A])
```

6. Expected Sarsa

6.1 Core Concept

Expected Sarsa is a **generalization** that uses the **expected value** of the next state under the current policy, rather than a sample (Sarsa) or the maximum (Q-Learning). It reduces variance by averaging over all possible next actions.

6.2 Update Rule

```
Q(S,A) <-- Q(S,A) + alpha * [R + gamma * sum_a pi(a|S') * Q(S',a) - Q(S,A)]
```

Where the expected value is: **E_pi[Q(S',A')] = sum_a pi(a|S') * Q(S',a)**

6.3 Computing the Expected Value Under Epsilon-Greedy

Under an epsilon-greedy policy with |A| actions:

Action Type	Probability	Example (epsilon=0.1, 4 actions)
Greedy action (argmax Q)	(1 - epsilon) + epsilon/|A|	0.9 + 0.1/4 = 0.925
Each non-greedy action	epsilon/|A|	0.1/4 = 0.025

Implementation of the expected value calculation:

```
# Build policy probability vector policy = np.ones(self.num_actions) * self.epsilon / self.num_actions best_action = self.argmax(self.q[state,:]) policy[best_action] = (1 - self.epsilon) + self.epsilon / self.num_actions # Compute expected Q-value expected_value = 0 for a in range(self.num_actions): expected_value += policy[a] * self.q[state, a] # Or equivalently using numpy: # expected_value = np.dot(policy, self.q[state, :])
```

6.4 Why Expected Sarsa is Better

Comparison	Expected Sarsa Advantage
vs Sarsa	Same convergence target but LOWER VARIANCE because it averages over all next actions instead of sampling one
vs Q-Learning	Can match Q-Learning's performance while being more stable; resistant to outlier random explorations
Step-size sensitivity	Most robust to step-size choice because lower variance means updates are more consistent

6.5 Complete Implementation

```
# In agent_step: S = self.prev_state A = self.prev_action state = observation # Choose next action (behavior policy) if self.rand_generator.rand() < self.epsilon: action = self.rand_generator.randint(self.num_actions) else: action = self.argmax(self.q[state,:]) # Build epsilon-greedy policy probabilities over next state policy = np.ones(self.num_actions) * self.epsilon / self.num_actions best_action = self.argmax(self.q[state,:]) policy[best_action] = (1 - self.epsilon) + self.epsilon / self.num_actions # Compute
```

```
expected value expected_value = sum(policy[a] * self.q[state, a] for a in
range(self.num_actions)) # Expected Sarsa update self.q[S, A] += self.step_size * ( reward +
self.discount * expected_value - self.q[S, A] ) # In agent_end (terminal state): S =
self.prev_state A = self.prev_action self.q[S, A] += self.step_size * (reward - self.q[S,
A])
```

7. Implementation Details & Common Pitfalls

7.1 The agent_info vs agent_init_info Bug

Notice in the original code template:

```
self.rand_generator = np.random.RandomState(agent_info["seed"])
```

This uses **agent_info** (the outer variable) rather than **agent_init_info** (the parameter). This is intentional in this template — agent_info is defined in the experiment cell as a global variable containing the seed. This pattern ensures reproducibility across runs.

7.2 State Visit Tracking (Experiment Cell)

The experiment tracks which states are visited in the last 10 episodes of each run. The key is using rl_start() and rl_step() manually instead of rl_episode():

```
state, action = rl_glue.rl_start() # Get initial state state_visits[state] += 1 # Count
initial state is_terminal = False while not is_terminal: reward, state, action, is_terminal
= rl_glue.rl_step() state_visits[state] += 1 # Count each visited state
```

7.3 The Q-table Structure

The Q-table is a 2D numpy array: **q[state, action]**

- Dimensions: (num_states=48) x (num_actions=4)
- Initialized to all zeros: np.zeros((self.num_states, self.num_actions))
- Index convention: state is the row, action is the column
- Actions: 0=up, 1=right (or similar — environment-specific)

7.4 Understanding prev_state and prev_action

TD algorithms are inherently one-step delayed: to update Q(S,A), you need the transition (S, A, R, S'). The agent stores the previous state/action to enable this:

```
# In agent_start: called at episode beginning self.prev_state = state # = first observation
self.prev_action = action # = first epsilon-greedy action chosen return action # In
agent_step: receives (R, S') and uses (prev_S, prev_A) # ... choose new action from S' ... #
Update: Q(prev_S, prev_A) using R and Q(S', new_action) self.prev_state = state # update for
next step self.prev_action = action
```

8. Experiment Results and Analysis

8.1 Episode Reward Plot (200 Episodes)

In the first experiment (200 episodes, alpha=0.5, epsilon=0.1), you should see:

Algorithm	Expected Behavior	Why
Expected Sarsa	Highest avg rewards (~-20 to -40)	Best balance: accounts for exploration, reduces variance
Sarsa	Medium avg rewards (~-30 to -50)	Takes safer path but accounts for exploration cost
Q-Learning	Lowest avg rewards (~-50 to -100)	Optimal path but suffers from cliff falls during exploration

8.2 State Visit Heatmaps

The heatmaps show which states are most frequently visited in the LAST 10 episodes (after learning has mostly converged). Expected patterns:

- **Sarsa heatmap:** Bright states along row above the cliff (safe path). The agent learned to avoid the cliff edge entirely.
- **Q-Learning heatmap:** Bright states along the cliff edge (bottom row). The agent learned the optimal path — right next to the cliff.
- **Expected Sarsa heatmap:** Similar to Q-Learning but possibly slightly above the cliff, or a mix — it finds the optimal path while accounting for exploration.

The gray area in the heatmaps represents the cliff itself (states that cause immediate reset). These cannot be 'visited' in the normal sense.

8.3 Step-Size Sensitivity Analysis (100 Episodes)

When varying alpha from 0.1 to 1.0 and averaging over 100 episodes:

- **All algorithms:** Peak performance around alpha=0.3-0.5
- **Q-Learning:** Severe performance degradation at alpha > 0.5. At alpha=1.0, performance can collapse completely because the updates are too large.
- **Sarsa:** Moderate sensitivity — degrades but more gradually than Q-Learning
- **Expected Sarsa:** Most robust — maintains good performance even at alpha=1.0 because the expected value reduces update variance

8.4 Step-Size Sensitivity Analysis (1000 Episodes)

With 1000 episodes (asymptotic performance):

Expected Sarsa and Sarsa show more similar performance for low alpha values — they both converge toward similar policies. Q-Learning still struggles at high alpha values. The key insight is that the performance ranking (Expected Sarsa > Sarsa > Q-Learning in terms of actual reward) holds across different step sizes because of the fundamental on-policy vs off-policy distinction.

9. Common Exam/Interview Questions

Q: What is the difference between on-policy and off-policy learning?

A: On-policy (e.g., Sarsa) evaluates and improves the same policy being used to make decisions. Off-policy (e.g., Q-Learning) evaluates a different (target) policy than the one generating behavior. Q-Learning learns the optimal policy even while following an exploratory epsilon-greedy policy.

Q: Why does Q-Learning perform worse than Sarsa in Cliff World during training?

A: Q-Learning learns that the cliff-edge path is optimal (it is, with a perfect greedy policy). However, because the ACTUAL behavior policy is epsilon-greedy (with random exploration), the agent sometimes randomly falls off the cliff, receiving -100 reward. Sarsa accounts for this exploration and learns a safer path away from the cliff.

Q: Why is Expected Sarsa better than both Sarsa and Q-Learning?

A: Expected Sarsa uses the expected Q-value over all actions (weighted by policy probabilities) rather than sampling one action (Sarsa) or taking the maximum (Q-Learning). This reduces variance in updates. It can match Q-Learning's convergence target when epsilon→0 while being more stable. It outperforms Sarsa because it avoids the stochastic noise of randomly sampling one action.

Q: What does the TD error represent, and why is it important?

A: The TD error delta = R + gamma*V(S') - V(S) represents the 'surprise' or 'prediction error' — how much the current estimate differs from what just happened. If delta > 0, the outcome was better than expected; if delta < 0, worse. TD errors are the learning signal and have biological correlates to dopamine neurons in the brain.

Q: What is bootstrapping in TD learning?

A: Bootstrapping means updating estimates based on other estimates, rather than waiting for the final outcome. In Sarsa: Q(S,A) is updated using Q(S',A') — another estimate. This contrasts with Monte Carlo methods which use actual returns. Bootstrapping allows learning before an episode ends but introduces bias.

Q: When would you prefer Q-Learning over Sarsa in practice?

A: Use Q-Learning when: (1) the agent won't actually follow epsilon-greedy at test time (only during training), (2) you want to learn the optimal policy value Q*, (3) safety during training is not a concern. Use Sarsa when the exploration policy IS the deployment policy, or when dangerous exploration must be avoided.

Q: What happens to Sarsa if epsilon → 0?

A: If epsilon → 0, Sarsa becomes equivalent to Q-Learning because the greedy action is almost always chosen, so Q(S',A') ≈ max_a Q(S',a). Sarsa and Q-Learning converge to the same optimal policy as epsilon decreases to 0.

Q: Why is tie-breaking important in argmax?

A: When Q-values are initialized to zero, ALL actions are tied. Without random tie-breaking, the agent would always break ties in favor of lower-indexed actions, creating systematic bias in exploration. Random

tie-breaking ensures fair exploration among equally-valued actions.

10. Final Algorithm Comparison

Property	Sarsa	Q-Learning	Expected Sarsa
Update target	R + g*Q(S',A')	R + g*max Q(S',a)	R + g*E[Q(S',a)]
Policy type	On-policy	Off-policy	On/Off-policy*
Convergence	Epsilon-greedy policy value	Optimal value Q*	Optimal value Q* (if epsilon->0)
Variance	High (1 sample)	High (max can be unstable)	Low (full average)
Computation	O(1)	O(|A|)	O(|A|)
Step-size sensitivity	Medium	High	Low
Cliff World behavior	Safe path	Risky optimal path	Near-optimal stable path
Best for	When exploration cost matters	When only test performance matters	General purpose; best overall

* Expected Sarsa with epsilon-greedy is technically on-policy during training, but when epsilon=0 it becomes equivalent to Q-Learning (off-policy). It generalizes both Sarsa and Q-Learning.

This lab implements foundational algorithms from Chapter 6 of Sutton & Barto's 'Reinforcement Learning: An Introduction'. These algorithms are the building blocks for modern deep RL methods like DQN (Deep Q-Network) and actor-critic methods used in ChatGPT (RLHF).